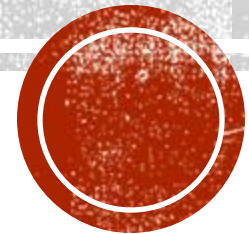


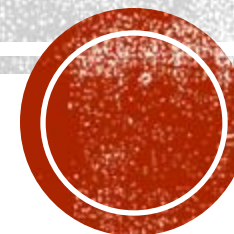
WEEK 9

Neural Network Libraries



NEURAL NETWORK

SK Learn Library



USING SKLEARN

- MLP

```
X=numpy.array([ [0, 0] , [0, 1], [1, 0], [1, 1] ])
```

```
y=[ 0, 1, 1, 0]
```

```
mlp = MLPClassifier()
```

```
mlp.fit(X, y)
```



LAYER

- Input layer(s)
 - Dimension of input layers
- Hidden layer(s)
- Output layer



USING SKLEARN

- `MLPClassifier`
 - `activation='tanh',`
 - `hidden_layer_sizes=(3,) → (3,2,5,)`
 - `solver='sgd'`
 - `learning_rate_init=0.01`
- `max_iter=500`



MLP PARAMETERS

```
▪ from sklearn.neural_network import MLPClassifier
```

Parameters

- hidden_layer_sizes=(3,)
- activation='logistic'
- learning_rate_init=0.2
- solver='sgd'
- max_iter=10000
- tol=0.000001
- verbose=1
- random_state=10)

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

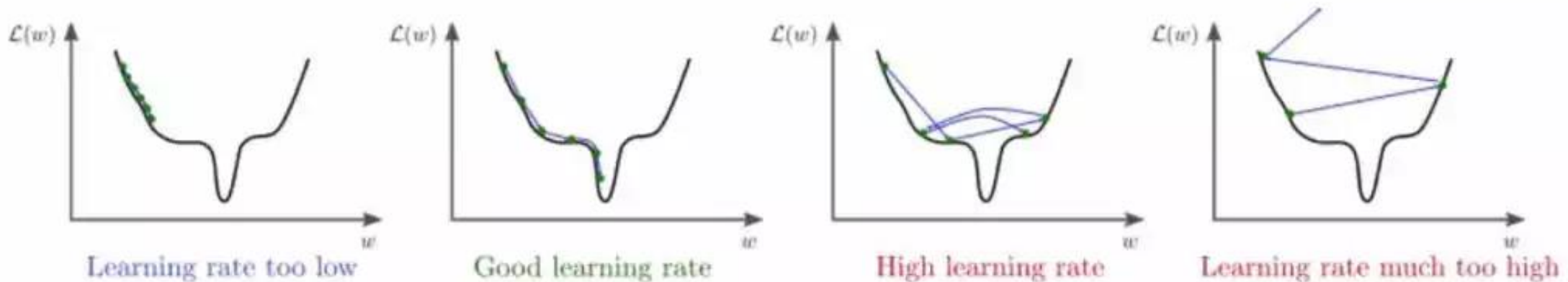
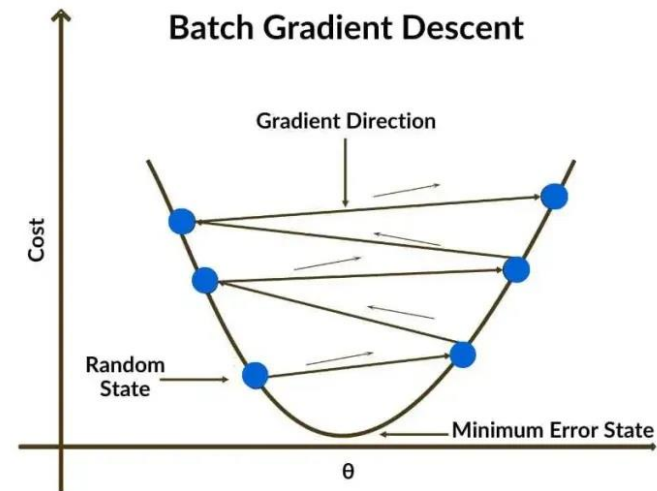


LEARNING PARAMETER

- **Training Method**
 - gradient descent, scaled conjugate gradient, adam,
- **Activation**
 - Relu, Softmax, relu, tanh
- **learning_rate**
 - Constant → Remain constant
 - Adaptive → smaller as time goes by (uses **power_t** to adjust)
 - Dyname → smaller as loss decreases
- **learning_rate_init** → Stating value
 - **Random_state** random_state
- **Batch_size**
 - Divide big data sizes to smaller batch (default =200)



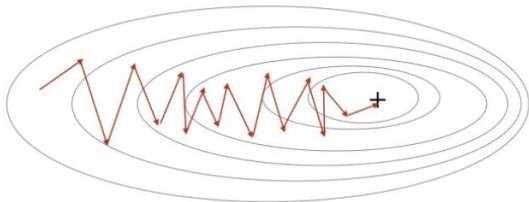
GRAD DESCEND AND STOCHASTIC CONCEPT



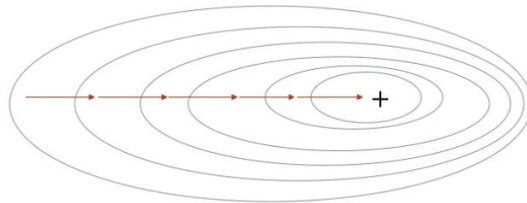
STOCHASTIC GRADIENT AND ADAM

Add minibatch

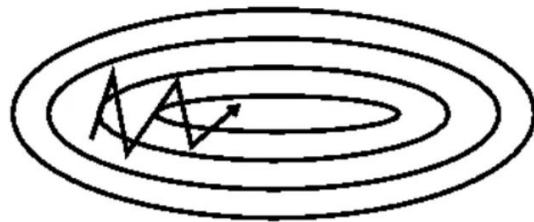
Stochastic Gradient Descent



Gradient Descent



- Add mini batch random
- Add momentum
- Add regularization → decay the term of the past gradient

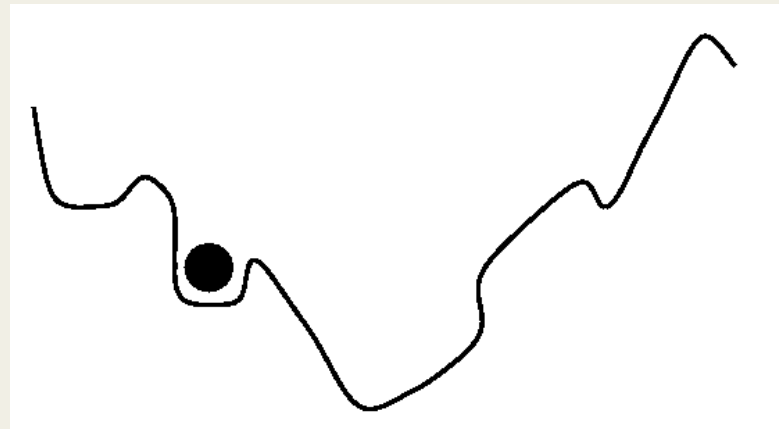
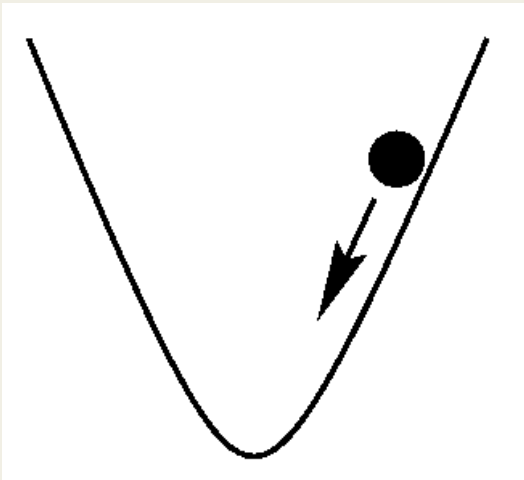


$$E(w, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(w)$$



USING MOMENTUM

- 🕒 Allow to go upward. (0- 1): Default 0.9 for sgd
- 🕒 Avoid local minima
- 🕒 Calculated by weighted average for Adam to converge easier



HYPER PARAMETER FOR TRAINING

- Training Method (Solver) *{'lbfgs', 'sgd', 'adam'}, default='adam'*
 - 'lbfgs' is an optimizer in the family of quasi-Newton methods.
 - 'sgd' refers to stochastic gradient descent.
 - 'adam' refers to a stochastic gradient-based optimizer proposed
- Learning rate(η) *{'constant', 'invscaling', 'adaptive'} → initial_learning_rate_init*
 - Constant → Remain constant
 - invscaling → smaller as time goes by (uses **power_t** to adjust)
 - adaptive → smaller as loss decreases
- Training iteration(epoch)
 - The maximum iteration that the training will stop if not converged



SKLEARN — IMPORTANT FUNCTIONS

- train data
 - `mlp.fit(x,y)`
- Get the weight
 - `mlp.coefs_`
- Accuracy
 - `mlp.score(X,y)`
- Predict the data
 - `mlp.predict(X)`



STOPPING CRITERIA

- **Iteration**

- `max_iter` = maximum iterations

- **Changes**

- `tol` = Error has not changed with in this (`loss > previous_loss - tol`)
 - Early stoping = stop if too has not changed for `n_iter_no_change`



GRID SEARCH

- Create parameter sets
 - `lr=[0.1,0.01,0.2]`
 - `layer=[(3,),(5,),(7,),(9,)]`
 - `solve=['sgd', 'adam']`
- Create Dic Grid
 - `param = dict(learning_rate_init=lr,hidden_layer_sizes=layer,solver=solve)`
Or create directly
 - `parameters = {'solver':('sgd', 'adam') }, 'learning_rate_inte':[0.1, 0.2,0.3]}`
- Do the grid search
 - `grid = GridSearchCV(estimator=network, param_grid=param,cv=3)`



RANDOMIZE SEARCH: EXAMPLE

```
parameter_space = { 'hidden_layer_sizes':  
    [(sp_randint.rvs(100,600,1),sp_randint.rvs(100,600,1)),  
     (sp_randint.rvs(100,600,1),)],  
    'activation': ['tanh', 'relu', 'logistic'],  
    'solver': ['sgd', 'adam', 'lbfgs'], 'alpha': uniform(0.0001, 0.9),  
    'learning_rate': ['constant', 'adaptive']}
```



PIPELINE

```
steps = [('scaler', StandardScaler()),  
          ('mlp', MLPClassifier())]
```

```
from sklearn.pipeline import Pipeline
```

```
pipeline = Pipeline(steps)  
# define the pipeline object.  
grid = GridSearchCV(pipeline, param_grid=parameteres,  
cv=5)
```



EXPERIMENT ON SIMPLE PICTURE DATA

- https://github.com/AjMing/Pattern-EGCI463/blob/main/Week7-8/Reading_Image_form_a_folder.ipynb
- Try simple MLP with a few layers
- You can use LDA or PCA to help with the pipeline

